Gradient Descent and XGBoost

Write a python program to predict a non-linear target variable y using two predictors x_1 and x_2.

$Y = 3x_1^2+2\sin(x_2) + €$

Where $€ \sim N(0, 2^2)$ is a random noise.

- **Implement Multiple Linear Regressions using Gradient Descent (not using built-in libraries like LinearRegression).**
- **Compare its performance with a nonlinear machine learning model (XGBoost) on the same dataset.**

Procedure:

Step 1: Import the required Python libraries such as NumPy, Pandas, Matplotlib, Scikit-learn, and XGBoost.

Step 2: Set the random seed and generate 500 observations for the predictors x1 and x2 using random values between -5 and 5.

Step 3: Generate the error term ϵ\epsilon from a normal distribution with mean 0 and standard deviation 2.

Step 4: Generate the nonlinear target variable yy using the given formula:

Step 5: Store x1, x2, and y in a Pandas DataFrame and display the first few observations.

Step 6: Divide the dataset into training and testing sets, using 80% for training and 20% for testing.

Step 7: Standardize the predictor variables using the mean and standard deviation of the training data.

Step 8: Add an intercept term and initialize the coefficients to zero for Multiple Linear Regression.

Step 9: Apply the Gradient Descent algorithm to estimate the regression coefficients by repeatedly updating them using the learning rate and calculated gradients. The assignment specifically requires Gradient Descent rather than a built-in LinearRegression model.

Step 10: Use the estimated coefficients to predict the yy values for the test data.

Step 11: Calculate the MSE, RMSE, and R2R^2 score for the Gradient Descent model.

Step 12: Build an XGBoost Regressor model using the same training data and generate predictions for the test data.

Step 13: Calculate the MSE, RMSE, and R2 score for the XGBoost model.

Step 14: Compare the performance of Gradient Descent and XGBoost using the calculated performance measures.

Step 15: Plot the Actual vs Predicted values for both models to visually compare their prediction performance.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from xgboost import XGBRegressor

# Generate nonlinear dataset

np.random.seed(42)
n = 500
x1 = np.random.uniform(-5, 5, n)
x2 = np.random.uniform(-5, 5, n)

# True nonlinear target
y = 3 * (x1 ** 2) + 2 * np.sin(x2) + np.random.normal(0, 2, n)

X = np.vstack((x1, x2)).T #hape should be (n, 2) instead of (2, n)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Linear Regression via Gradient Descent
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Add intercept (bias term)
Xb_train = np.c_[np.ones(X_train_scaled.shape[0]), X_train_scaled]
Xb_test = np.c_[np.ones(X_test_scaled.shape[0]), X_test_scaled]

# np.ones(X_train_scaled.shape[0]) → creates a column of 1's, shape (n_samples, 1).
# X_train_scaled → shape (n_samples, n_features).
# np.c_[] → combines them into a single matrix with shape (n_samples, n_features+1).

# Initialize weights
w = np.zeros(Xb_train.shape[1])

# Hyperparameters
```

```
alpha = 0.05    # learning rate
epochs = 1000
m = len(y_train)
losses = []

for epoch in range(epochs):
    # Predictions
    y_pred = np.dot(Xb_train, w)
    # Gradient
    grad = (1/m) * np.dot(Xb_train.T, (y_pred - y_train))
    # Update weights
    w -= alpha * grad
    # Loss
    loss = (1/(2*m)) * np.sum((y_pred - y_train)**2)
    losses.append(loss)

y_pred = np.dot(Xb_train, w)

grad = (1/m) * np.dot(Xb_train.T, (y_pred - y_train))

w -= alpha * grad

# Predictions on test data
y_pred_lr = np.dot(Xb_test, w)

mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

# Convert weights back to original scale -----------------------------------------
# Gradient descent then found weights w0, w1, w2 corresponding to the scaled features.
# But those weights are not interpretable in the original feature scale.
# To report the regression coefficients in the original data units,
# we must "undo" the scaling.

beta1 = w[1] / scaler.scale_[0]
beta2 = w[2] / scaler.scale_[1]
intercept = w[0] - (beta1 * scaler.mean_[0] + beta2 * scaler.mean_[1])

# XGBoost

xgb_model = XGBRegressor(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=4,
    reg_lambda=1,
    random_state=42
)
```

```
xgb_model.fit(X_train, y_train)

y_pred_xgb = xgb_model.predict(X_test)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)

# Results

print("Linear Regression (Gradient Descent):")
print(f"Equation: y = {intercept:.3f} + {beta1:.3f}*x1 + {beta2:.3f}*x2")
print(f"Test MSE: {mse_lr:.4f}, R²: {r2_lr:.4f}\n")

print("XGBoost:")
print(f"Test MSE: {mse_xgb:.4f}, R²: {r2_xgb:.4f}")

# Plots

# Loss curve

plt.figure(figsize=(12,4))

plt.plot(losses)

plt.title("Gradient Descent Loss Curve (Linear Regression)")
plt.xlabel("Epochs")

plt.ylabel("Loss (MSE)")

plt.show()

# Predictions vs Actual
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)

plt.scatter(y_test, y_pred_lr, alpha=0.6, color="red")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "k--")
plt.title("Linear Regression (Gradient Descent)")

plt.xlabel("Actual y")

plt.ylabel("Predicted y")

plt.subplot(1,2,2)

plt.scatter(y_test, y_pred_xgb, alpha=0.6, color="green")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "k--")
plt.title("XGBoost Predictions")
```

```
plt.xlabel("Actual y")

plt.ylabel("Predicted y")

plt.tight_layout()

plt.show()
```

Output:

Linear Regression (Gradient Descent):

Equation: y = 26.881 + 0.085*x1 + -0.003*x2

Test MSE: 589.3888, R^2: -0.0037

XGBoost:

Test MSE: 6.2443, R^2: 0.9894

